

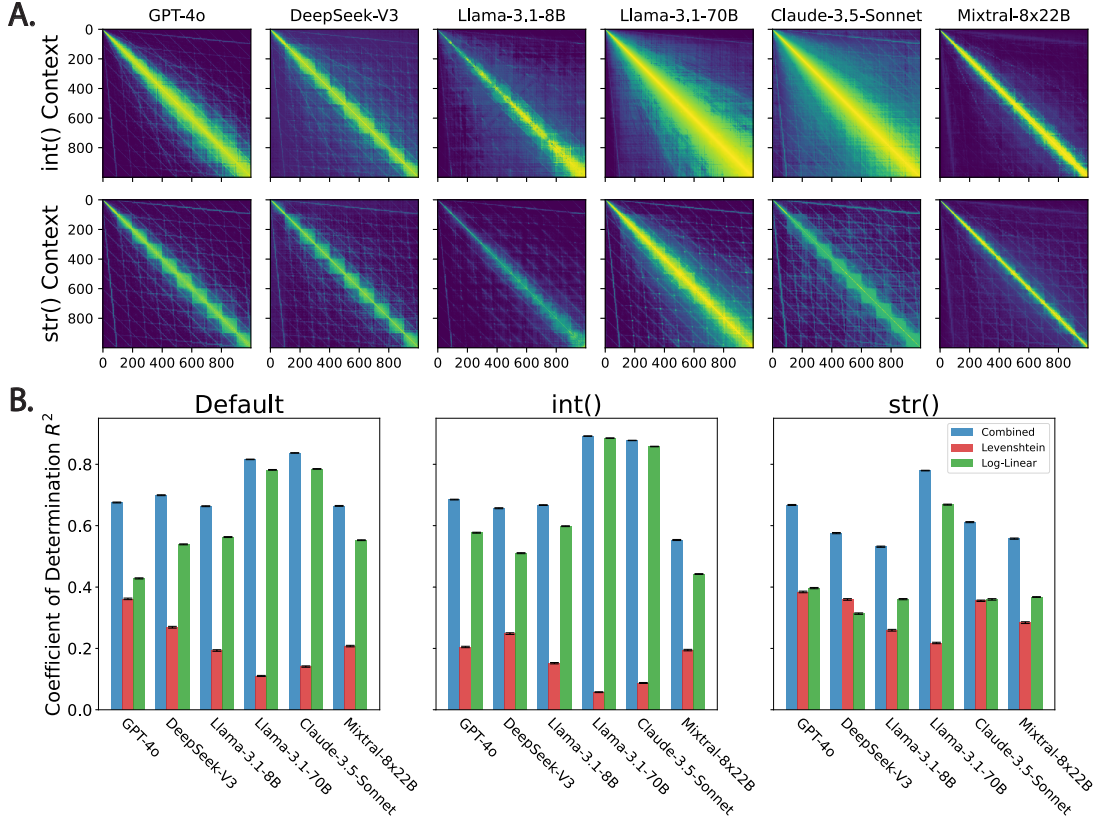


Figure 2. Context effects on LLM-elicited number similarity matrices and their decomposition. **A.** LLM similarity matrices under the effect of ‘type’ specification: `int()` vs. `str()` (see Appendix A for prompts). **B.** Coefficient of determination (R^2) for the different similarity matrices under the default (Figure 1), `int()`, and `str()` contexts for the combined and separate Levenshtein (string) and Log-Linear (numerical) distance predictors (error bars indicate 95% confidence intervals; see Methodology).

values against two theoretical distance measures, namely, the Levenshtein string edit-distance $d_{Lev}(a, b)$ (Levenshtein, 1966) and the psychological Log-Linear number distance $d_{Log}(a, b)$ (Piantadosi, 2016) (we also considered a simple linear ℓ_1 distance $|a - b|$ but found that it led to worse results; Appendix Figure 6). The Log-Linear distance captures the psychological phenomenon whereby humans represent larger numbers with less fidelity leading to a logarithmic sensitivity to magnitude, i.e. $x - y \rightarrow \log(x) - \log(y)$ (see Appendix B for definition). The Levenshtein distance, on the other hand, is defined as the minimum number of deletions, insertions, or substitutions required to transform one string to the other. This can be defined recursively (see Appendix B) and we used the Python package `Levenshtein`¹ to compute it. For example, the Levenshtein distance between the digits 200 and 100 is 1 because it takes a single substitution ($2 \rightarrow 1$) to transform one string into the other.

Given the two distance measures evaluated on the range of interest $d_{ij}^{(Lev)}$ and $d_{ij}^{(Log)}$, we transformed them into similar-

ity scores $s_{ij}(d) = 1 - d_{ij} / \max\{d_{ij}\}$ and then fitted them to s_{ij} (after z-scoring), i.e., $s_{ij} = \alpha + \beta s_{ij}^{(Lev)} + \gamma s_{ij}^{(Log)}$ using the `LinearRegression` method from `scikit-learn` (Pedregosa et al., 2011) and computed the coefficient of determination (R^2 or variance-explained) which we also repeated for the individual metrics separately. To derive confidence intervals, we bootstrapped over the number pairs (since the models had zero temperature) with replacement and 1,000 repetitions. All analysis code is provided in the accompanying repository (see Reproducibility).

4.4. Probing Language Models

When generating an output, a language model first tokenizes the input sentence into a sequence of tokens $x_1, \dots, x_n \in V$ where V is some vocabulary. Then during the forward pass, the model incrementally transforms the initial embeddings (extracted from a vocabulary embedding layer) through each layer. The internal representations at each layer — commonly referred to as *residuals* — capture evolving latent features. These residuals can be analyzed to probe specific properties of the input or model behavior.

¹<https://rapidfuzz.github.io/Levenshtein/>

In our case, we train an affine transformation for each layer within the model that takes in a specific latent $h_i^{(j)}$ for token i at layer j , and then learns a vector w and bias term b , such that $w \cdot h_i^{(j)} + b$ predicts a specific distance measure. Due to computational constraints, the probes are run only on the Llama-3.1-8b model. We take the original similarity prompt and extract the residuals from the last token — in our case the “.” after “Result:” (see Appendix A) — and train two linear probes on them. One probe is trained to predict the Levenshtein distance between the two inputted numbers, the other the Log-Linear distance.

We train the probes on 9,500 random pairs of numbers between 0 and 999, and evaluate on the range 0-500. We ablate across all layers and select layer 25 for the final probe (see Appendix D for ablations on number of training points and performance across layers). We then run these probes on all 500×500 (250,000) combinations of number similarities and report their corresponding correlations. To illustrate the results of the probe, we apply multidimensional scaling (MDS) (Shepard, 1962) on the 500×500 similarity judgments extracted by the probe. MDS is a visualization technique that represents high-dimensional data in a lower-dimensional space, preserving the relative pairwise distances or dissimilarities as closely as possible. We computed MDS embeddings using `scikit-learn`.

4.5. Constructing Close Triplets

Our goal was to construct diagnostic triplets for the naturalistic decision scenario such that (i) the options are numerically close to each other but there is an unambiguous correct answer, and (ii) the options are very different in terms of their edit distance relative to the target. To do that, we constructed the target quantity q_0 by randomly sampling three digits from the set $\{2, \dots, 9\}$ with replacement and combining them into a number (e.g., 3, 3, 1 $\rightarrow$ 331). Then, we constructed a Levenshtein-aligned option by subtracting 1 from the largest decimal entry (i.e., 331 $\rightarrow$ 231). Finally, we constructed a Log-aligned option by keeping the largest decimal entry from the target and randomly sampling two new integers from the range $\{1, \dots, 9\}$ excluding the digits that appeared in the other two numbers (e.g., 357). Thus, the result would be (331, 231, 357) in our example which satisfies the desired properties. We repeated the process also for five digit numbers to probe how things change for longer numbers (e.g., 25337, 15337, 26886). We sampled 10,000 such triplets from each length category and took the unique subset. Overall, there were 6,474 unique three digit triplets, and 9,995 five digit triplets.

5. Experiments

5.1. Eliciting Similarity Judgments

In the first experiment, we elicited similarity judgments across all pairs of integers in the range 0 – 999 (‘How similar are the two numbers?’; see Appendix A for the full prompt). We begin by presenting the qualitative patterns in the raw data and then quantify them using a suitable regression analysis. Figure 1 shows the similarity matrices for all models (see Methodology). The resulting LLM matrices are highly structured with a dominant area around the diagonal that in some cases (e.g., GPT-4o and DeepSeek-V3) takes a block-diagonal form, in addition to other sub-diagonal structures. These patterns appear to blend the properties found in two theoretical similarity matrices associated with string edit distance (Levenshtein) and numerical distance (Log-Linear) highlighted in the rightmost panels of Figure 1 (see Methodology). To further see whether these patterns arise from a tension between string and integer representations, in a following experiment we resolved the ambiguity of the similarity prompt by specifying the ‘type’ of the token using `int()` and `str()` contexts (see Appendix A for prompts). The results are shown in Figure 2A. In this case, we found that the context intervention pushed the matrices in opposing directions. For example, GPT-4o loses its block diagonal structure in the `int()` context, whereas Claude-3.5-Sonnet gains it in the `str()` context. Similar trends can also be found in the other models.

To quantify the above, we regressed the Levenshtein and Log-Linear distance measures against the LLM similarity judgments in the three contexts considered. A breakdown of the explained variance (coefficient of determination R^2) per condition are provided in Figure 2B. Overall, in the default context we found that a linear combination of the numerical and Levenshtein distance measures is sufficient to achieve an average R^2 of .726 (95% CI of mean: [.725, .726]), with the separate components each contributing significantly (Log-Linear only R^2 CI: [.607, .609], Levenshtein only R^2 CI: [.213, .215]). We note that replacing the Log-Linear distance with a simple linear ℓ_1 distance diminishes combined average performance to an R^2 of .567 (95% CI: [.567, .568]; see Appendix Figure 6). In the other contexts, the metrics were correspondingly pushed in opposing directions. In the `str()` case, the mean R^2 CIs were: Combined: [.620, .621], Log-Linear: [.410, .412], and Levenshtein: [.309, .311]. On the other hand, in the `int()` context these were: Combined: [.721, .722], Log-Linear: [.645, .646], and Levenshtein: [.156, .158].

Next, we evaluated the extent to which the string-like similarity generalizes to less common number bases. The idea here is that if the model is relying on edit distance, then the effect should persist irrespective of how uncommon the underlying numerical basis is. To that end, we elicited ad-

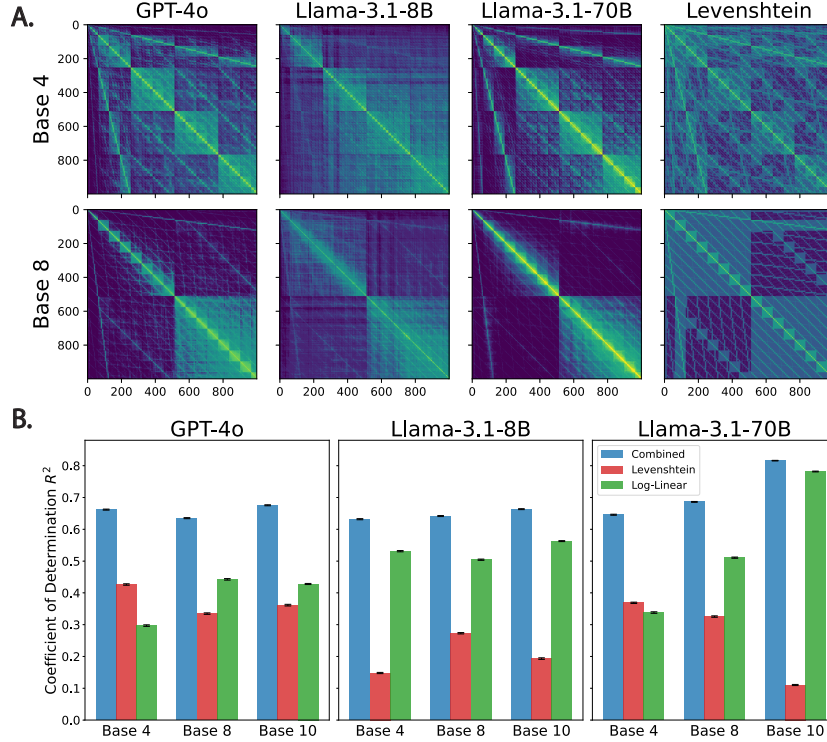


Figure 3. The effect of other number bases on elicited similarity. **A.** LLM similarity matrices over all integer pairs in the range 0 – 999 represented in base 4 and 8 along with the corresponding Levenshtein distance measures (see Appendix A for prompts). **B.** Coefficient of determination (R^2) for the various similarity matrices under the different base contexts (including the base 10 results from Figure 1) for the combined and separate Levenshtein (string) and Log-Linear (numerical) distance predictors (error bars indicate 95% CIs).

ditional similarity judgments for a subset of three models (GPT-4o, Llama-3.1-8b, and Llama-3.1-70b) using two additional number bases: base 4 and base 8 (see Methodology). The results are shown in Figure 3A. Here too we found highly structured matrices with patterns that are consistent with those derived from the Levenshtein distance. Quantitatively, we found that all bases resulted in a significant Levenshtein contribution (Figure 3B), and in the case of GPT-4o and Llama-3.1-70b those contributions were enhanced relative to the Log-Linear metric. Indeed, compare the R^2 CIs for the Log-Linear vs. Levenshtein regressors: In the case of Llama-3.1-70b, for base 10 we had: Log-Linear: [.780, .783] and Levenshtein: [.108, .112], whereas for base 4 this is reversed: Log-Linear: [.336, .340], and Levenshtein: [.366, .371]. Likewise, for GPT-4o in base 10: Log-Linear: [.426, .430], and Levenshtein: [.359, .364], whereas for base 4: Log-Linear: [.295, .299], and Levenshtein: [.424, .428].

5.2. Probing Internal Representations

The previous experiments showed that when an LLM is prompted for similarity judgments between two different integer tokens, the resulting behavioral metric is a combination of string and integer distance. In this section, we

show how this also holds on the representational (i.e., token-embedding) level in a model for which we have internal access (Llama-3.1-8b). To do that, we train linear probes from the last token in the prompt to decode the Log-Linear and Levenshtein distances between the integers in question (see Methodology). This is non-trivial as there is no guarantee that such a linear transformation exists unless the model encodes the relevant information in its embedding (this can be seen also from the fact that our probes had 4,096 parameters, corresponding to the latent dimension, fitted on 9,500 data points, and then evaluated on 250,000 data points; see Methodology). In Figure 4 we illustrate the decoded similarity matrices from the embeddings. Here we notice a pattern that is consistent with the behavioral (prompt-based) data. In the string probe case (right panel in Figure 4), the model is able to capture the edit-distance similarity between two numbers; whereas in the integer probe case (left panel in Figure 4), the decoded pattern is indeed much more log-linear as can be seen from the diagonal area in the matrix. Interestingly, however, string similarities still bleed into the representation as can be seen from the sub-diagonal structures. Quantitatively, the Pearson correlation between the string probe and the Levenshtein and Log-Linear measures is .650 and .527, respectively. For the integer probe, the